GRAND CHALLENGE MONOGRAPH COLLECTION

TYPE THEORY

The Grand Unified Theory of Computation

INSTRUCTOR AND SELF-STUDY SOLUTIONS COMPANION

Volume I: Judgment

Grand Challenge Labs

First Edition — Release Candidate 1 — 2026

How to Use This Companion

This companion addresses every one of the 168 chapter exercises in Volume I. It is an answer key and teaching aid, not a substitute for doing the derivations. For open-ended Design Clinic and Challenge problems, the entry gives a reference design or grading rubric rather than pretending that one wording is uniquely correct.

Difficulty is calibrated by exercise mode: ⋆ Checkpoint, ⋆⋆ Core, ⋆ ⋆ ⋆ Synthesis and Proof Workshop, ⋆ ⋆ ⋆⋆ Design Clinic, and ⋆ ⋆ ⋆⋆–⋆ Challenge. A few chapter-specific outliers are identified in the calibration appendix.

All theorem references point back to the main Volume I manuscript. The solutions use the same metatheoretic convention: fresh formal binders, capture-avoiding substitution, and the TinyTT simple core unless stated otherwise.

Contents

Chapter 1

Before Programs: What Does It Mean for an Expression to Make Sense?

Checkpoint

Checkpoint 1 ⋆ *Explain, without using the word "error," why* `true + 3` *may be grammatical but not typable.*

The grammar may admit a binary addition node with any two expressions as children. Typing adds the stronger premise that both operands have the numeric type expected by addition. Thus the expression can have a syntax tree while no typing derivation for it exists.

Checkpoint 2 ⋆ *Give an example of a physical action on a calculator that does not correspond to a meaningful mathematical expression.*

For example, press an operator before supplying an operand, or enter a partially completed expression and stop. The key presses are physical events accepted by the hardware, but there need not be a completed mathematical expression corresponding to the resulting state.

Checkpoint 3 ⋆ *In the judgment* $x : \mathbb{N} \vdash x + 1 : \mathbb{N}$*, identify what is assumed and what is claimed.*

The context assumes that x is available as a natural number. The conclusion claims that, under that assumption, the term $x + 1$ is a natural number.

Core

Core 1 ⋆⋆ *Invent a tiny language with two base types. State three expressions that parse but should not type-check.*

One example uses base types Nat and Bool with expression grammar $e ::= 0 \mid \text{true} \mid \text{succ}(e) \mid \text{not}(e)$. Then succ(true), not(0), and succ(not(true)) all parse but fail the evident typing premises.

Core 2 ⋆⋆ *Explain why interpreting every type as a set at the outset may hide distinctions relevant to computation.*

A set model identifies a type primarily with its extension. That can hide operational distinctions such as evaluation strategy, linear use, protocol order, effects, proof relevance, or which eliminations are computationally available. Set semantics is valuable, but it is one semantics of the rules rather than the rules themselves.

Core 3 ⋆⋆ *Write four different English readings of* $\Gamma \vdash a : A$*: one as a programmer, one as a logician, one as a protocol designer, and one as a compiler engineer.*

Programmer: in environment Γ, a satisfies interface A. Logician: from assumptions Γ, a is evidence for proposition A. Protocol designer: given protocol state/capabilities Γ, payload or action a is admissible under contract A. Compiler engineer: the static analysis has constructed a typing derivation assigning A to a under symbol table Γ.

Synthesis

Synthesis 1 ⋆⋆⋆ *A spreadsheet cell may contain a number, formula, date, or text. Analyze a spreadsheet as a primitive typed language. Where does the analogy succeed, and where does it fail?*

The analogy succeeds because cells have distinguishable kinds, formulas have formation rules, references create dependencies, and operations are admissible only for compatible data. It fails because ordinary spreadsheets often use coercions, dynamic errors, mutable global recalculation, and ad hoc formats rather than a clean static judgment. A spreadsheet is therefore a suggestive weakly typed language, not a canonical type theory.

Synthesis 2 ⋆ ⋆ ⋆ *Argue for or against: "A parser decides whether a sentence exists; a type checker decides whether it says something." Refine the sentence until you are satisfied with it.*

The slogan is too strong. A parser establishes that a token sequence belongs to a syntactic grammar; it does not decide metaphysical existence. A type checker establishes a particular static judgment relative to a type system; it does not decide all meaning. A better sentence is: "Parsing constructs an expression licensed by the syntax; type checking establishes one formally specified class of semantic admissibility for that expression."

Proof Workshop

Proof Workshop 1 ⋆ ⋆ ⋆ *Formalize a grammar in which both* `succ 0` *and* `succ true` *parse. Then state the typing premise that distinguishes them.*

Take the grammar $e ::= 0 \mid \mathsf{true} \mid \mathrm{succ}(e)$. Both terms parse by the same succ production. The typing rule is $\frac{\Gamma \vdash e : \mathbb{N}}{\Gamma \vdash \mathrm{succ}(e) : \mathbb{N}}$, so succ(0) is derivable and succ(true) is not.

Proof Workshop 2 ⋆ ⋆ ⋆ *Prove that "parses" does not imply "has type* $\mathbb{N}$*" by giving a concrete counterexample in TinyTT.*

TinyTT parses `succ true` into a Succ syntax node. Its checker then requires the child to check at $\mathbb{N}$; `true` instead synthesizes Bool. Hence parsing succeeds while the judgment $\vdash \mathrm{succ}(\mathsf{true}) : \mathbb{N}$ has no derivation.

Design Clinic

Design Clinic 1 ⋆ ⋆ ⋆⋆ *Design a three-stage diagnostic for a tiny language that separately reports tokenization failure, parse failure, and typing failure. Give one input that reaches each stage.*

Use three tagged phases. Lexing reports an unrecognized character sequence, e.g. `@0`; parsing reports a token stream that violates the grammar, e.g. an unfinished conditional; typing reports a valid AST whose rule premises fail, e.g. `succ true`. Preserve the phase and source span in the diagnostic so later errors are never misreported as earlier ones.

Challenge

Challenge 1 ⋆ ⋆ ⋆ ⋆ ⋆ *Can an untyped lambda term be meaningful? Give an answer that preserves the usefulness of type theory without declaring untyped computation meaningless by fiat.*

Yes. Untyped lambda calculus has a precise syntax and reduction semantics, so its terms can denote algorithms and support rigorous reasoning. Type theory contributes a stronger discipline: it classifies terms, rules out specified bad interactions, supports compositional invariants, and can internalize proofs. The defensible claim is not that untyped computation is meaningless, but that typing makes important dimensions of computational meaning explicit and checkable.

Chapter 2

Judgments: The Atomic Form of Formal Meaning

Checkpoint

Checkpoint 1 ⋆ *What is the difference between the expression $A \to B$ and the judgment $\Gamma \vdash A \to B$ type?*

$A \to B$ is an object-language expression. $\Gamma \vdash A \to B$ type is a meta-level judgment asserting that the expression is a legitimate type under the assumptions in Γ.

Checkpoint 2 ⋆ *Draw a derivation of $\vdash \mathrm{succ}^3(0) : \mathbb{N}$.*

Apply zero introduction once and successor introduction three times: $\vdash 0 : \mathbb{N}$, then $\vdash \mathrm{succ}(0) : \mathbb{N}$, then $\vdash \mathrm{succ}^2(0) : \mathbb{N}$, then $\vdash \mathrm{succ}^3(0) : \mathbb{N}$. The derivation tree is a four-node spine.

Checkpoint 3 ⋆ *Name the four recurring questions associated with a type former.*

Formation: when is the type legitimate? Introduction: how is canonical data of the type built? Elimination: how may such data be observed or used? Computation: what happens when an eliminator meets a corresponding constructor?

Core

Core 1 ⋆⋆ *Design introduction and elimination rules for a two-valued boolean type.*

Introduce constants $\mathsf{true}, \mathsf{false} : \mathsf{Bool}$. A useful eliminator is $\mathsf{if}(c; t; u)$ with premises $c : \mathsf{Bool}$ and both branches of a common type A, concluding type A.

Core 2 ⋆⋆ *State a plausible computation rule for your boolean eliminator.*

The computation equations are $\mathsf{if}(\mathsf{true}; t; u) \to t$ and $\mathsf{if}(\mathsf{false}; t; u) \to u$. They state exactly how elimination recovers the branch selected by the constructor.

Core 3 ⋆⋆ *Give an example showing that a well-typed term need not be canonical.*

$(\lambda x.x)\,0$ is well typed at $\mathbb{N}$ but is not canonical: it is an application redex. It reduces to the canonical numeral 0.

Synthesis

Synthesis 1 ⋆ ⋆ ⋆ *Explain why an API is partly characterized by its elimination rules: the observations or operations clients are permitted to perform.*

An API is defined not only by which values can be created but by which observations clients are licensed to make. Projection methods characterize products; calling characterizes functions; exhaustive case analysis characterizes sums. Elimination rules are therefore the formal analogue of a client's legal interface to an abstract value.

Synthesis 2 ⋆ ⋆ ⋆ *Compare a derivation tree with a provenance graph in a data-processing pipeline.*

A derivation tree records which rule instances justify a conclusion; a provenance graph records which transformations and inputs produced a data artifact. Both make dependency inspectable and support local diagnosis. The analogy breaks because derivations are usually proof objects in a formal calculus, whereas provenance graphs may record empirical or effectful processes that do not imply logical validity.

Proof Workshop

Proof Workshop 1 ⋆ ⋆ ⋆ *Prove the cited Proposition in Volume I again by induction on derivation height rather than last-rule inspection. Which proof is more informative here?*

Define height of an axiom as 1 and height of a rule node as $1 + \max$ of premise heights. Induct on the height of a derivation ending in $\Gamma \vdash \mathsf{succ}(n) : \mathbb{N}$. The final rule cannot be zero introduction or any rule whose conclusion has another outer term constructor; the only compatible final rule is successor introduction, yielding the premise $\Gamma \vdash n : \mathbb{N}$. Last-rule inspection is shorter because the syntax of the conclusion already determines the rule.

Proof Workshop 2 ⋆ ⋆ ⋆ *Define a derivation-height function and show that every premise of a finite derivation has strictly smaller height than its conclusion node.*

For a finite derivation tree, set $h(D) = 1$ at leaves and $h(D) = 1 + \max_i h(D_i)$ at an inference with premises D_i. Then for every premise, $h(D_i) \leq \max_j h(D_j) < 1 + \max_j h(D_j) = h(D)$. This gives the well-founded measure needed for induction on derivations.

Design Clinic

Design Clinic 1 ⋆ ⋆ ⋆⋆ *Construct a deliberately bad natural-number rule that makes inversion ambiguous. Explain which checker diagnostic becomes less precise as a result.*

For example add an unsound rule $\frac{\Gamma \vdash b : \mathsf{Bool}}{\Gamma \vdash \mathsf{succ}(b) : \mathbb{N}}$. Now a term headed by succ may have arisen from ordinary Nat-successor typing or from the bad Bool rule, so inversion no longer tells the checker which premise type the child must have. Diagnostics become ambiguous because the outer constructor no longer determines a unique typing obligation.

Challenge

Challenge 1 ⋆ ⋆ ⋆ ⋆ ⋆ *Suppose we add a rule with no premises concluding $\Gamma \vdash a : A$ for every a and A. What collapses? Give the computational as well as logical consequences.*

The premise-free rule derives every typing judgment. Computationally, the checker accepts applications, projections, cases, and recursors with incompatible operands, so progress can fail immediately. Logically, every proposition-as-type becomes inhabited, including $\mathbf{0}$; from an inhabitant of $\mathbf{0}$, the logic is inconsistent and every proposition follows.

Chapter 3

Contexts: Worlds of Assumption

Checkpoint

Checkpoint 1 ⋆ *Explain why $x + 1$ cannot be assigned a type without information about x.*

The variable is an open input. Without a declaration for x, the system does not know whether numeric operations on it are licensed; x could denote a function, boolean, resource, or nothing at all.

Checkpoint 2 ⋆ *State the context-formation rule in words.*

The empty context is well formed. If Γ is well formed, A is a well-formed type in Γ, and x is fresh, then extending the context with $x : A$ yields a well-formed context.

Checkpoint 3 ⋆ *What computational privileges correspond to weakening and contraction?*

Weakening permits an available variable to go unused; contraction permits a variable to be used more than once. Operationally these correspond roughly to discarding and duplicating access to information or a capability.

Core

Core 1 ⋆⋆ *Give a context whose declarations cannot be reordered once dependent types are admitted.*

Once dependency exists, $n : \mathbb{N},\ xs : \mathrm{Vec}(A, n)$ is well formed, while the reversed order is not: the declaration of xs mentions n, so n must already be available.

Core 2 ⋆⋆ *Construct a term that uses a variable twice and explain why contraction is implicated.*

$\lambda x.\langle x, x\rangle$ uses x twice. Its derivation relies on ordinary contexts allowing the same assumption to support two subderivations; a linear discipline would require explicit duplication or prohibit the term.

Core 3 ⋆⋆ *Construct a term whose type derivation uses weakening.*

$\lambda x.0$ at type $A \to \mathbb{N}$ introduces $x : A$ but the body does not use it. The derivation is possible because weakening allows the unused declaration.

Synthesis

Synthesis 1 ⋆⋆⋆ *Treat a shell session as a context. Which facts and capabilities accumulate as commands execute?*

A shell session accumulates a current directory, environment variables, files, credentials, process identifiers, aliases, and open resources. This resembles context extension because later commands are meaningful relative to accumulated state. It is only an analogy: shell state changes temporally and destructively, while a proof context is normally a static assumption structure.

Synthesis 2 ⋆ ⋆ ⋆ *Explain how a security capability system can be seen as a context discipline.*

A capability context lists which resources or authorities are available. Typing a command under that context can require possession of a file handle, network token, or privilege. Removing weakening/contraction for selected capabilities can prevent silent discard or duplication of linear resources.

Proof Workshop

Proof Workshop 1 ⋆ ⋆ ⋆ *Prove weakening for the variable and application cases of the core calculus in full detail.*

Variable case: if $x : A \in \Gamma$, the same declaration remains present after inserting fresh independent declarations, so the variable rule re-applies. Application case: from $\Gamma \vdash f : A \to B$ and $\Gamma \vdash a : A$, the induction hypotheses give both judgments in the weakened context; application elimination then derives $fa : B$ there.

Proof Workshop 2 ⋆ ⋆ ⋆ *Give an example showing why exchange needs a dependency side condition once declarations such as $y : B(x)$ are allowed.*

With $x : A$, $y : B(x)$, exchanging the declarations would require forming $y : B(x)$ before x exists. Exchange is valid only when later declarations do not depend on the declaration moved across them (or when the dependency is transported appropriately).

Design Clinic

Design Clinic 1 ⋆ ⋆ ⋆⋆ *Redesign TinyTT contexts so shadowing is impossible internally while user-facing names remain readable. State the invariant your representation enforces.*

Internally assign each binder a unique identifier, for example a monotonically allocated integer, and store display names separately. The invariant is: every context entry has a globally unique binder ID, and variable occurrences resolve to IDs rather than strings. Pretty-printing may reuse readable source names, but lookup and alpha-equivalence cannot be confused by shadowing.

Challenge

Challenge 1 ⋆ ⋆ ⋆ ⋆ ⋆ *Imagine a language in which variables must be used exactly once. Which familiar programming patterns become awkward? Which resource-sensitive patterns become easier to reason about?*

Exactly-once use makes ordinary duplication, dropping values, sharing caches, and many convenience APIs awkward unless explicit structural operations are provided. In return, ownership transfer, file handles, memory regions, channels, and other consumable resources become easier to reason about: the type discipline can guarantee that a resource is neither silently copied nor forgotten.

Chapter 4

Variables and Dependency: Naming What the Future May Supply

Checkpoint

Checkpoint 1 ⋆ *Identify the free variables of* $(\lambda x.\, f\, x)\; y$.

The free variables are $\{f, y\}$. The occurrence of x is bound by the lambda.

Checkpoint 2 ⋆ *Why are* $\lambda x.x$ *and* $\lambda z.z$ *considered the same binding structure?*

Renaming a bound variable consistently does not change which occurrences are bound or how the term computes. Both terms have one binder whose body is exactly the variable bound by that binder; they are alpha-equivalent.

Checkpoint 3 ⋆ *Explain a variable as a promise rather than an unknown.*

A variable records a slot that some admissible future value may fill. Reasoning with $x : A$ means proving that the construction works uniformly for any value supplied at that slot, rather than guessing a hidden concrete value.

Core

Core 1 ⋆⋆ *Give an example of variable capture caused by careless renaming.*

If one naively renames the outer binder in $\lambda x.y$ to y, the result $\lambda y.y$ captures the formerly free y. The binding graph changes, so this is not alpha-renaming.

Core 2 ⋆⋆ *Draw scopes for* $\lambda x.\lambda y.x$ *and* $\lambda x.\lambda y.y$.

In $\lambda x.\lambda y.x$, the outer scope contains the entire inner lambda and the occurrence of x points to the outer binder; the inner y is unused. In $\lambda x.\lambda y.y$, the final occurrence points to the inner binder and the outer x is unused.

Core 3 ⋆⋆ *Describe, in plain language, what a type family $B(x)$ does.*

A family $B(x)$ assigns a type to each admissible value of $x : A$. Learning or choosing x therefore determines which type describes the next object. A vector family $\mathrm{Vec}(A, n)$ is the standard example.

Synthesis

Synthesis 1 ⋆ ⋆ ⋆ *Compare lexical scope in programming languages with the visibility of assumptions inside a mathematical proof.*

Lexical scope determines where a program variable may be referenced; proof scope determines where an assumption may be used. Both are tree-structured disciplines of availability. Proof contexts additionally carry logical/type information, while programming scopes may contain mutable bindings and effects.

Synthesis 2 ⋆ ⋆ ⋆ *Explain why dependency makes context order significant.*

A later declaration may mention an earlier variable, so reordering can make that mention unbound. Dependency turns context order from presentation into well-formedness data.

Proof Workshop

Proof Workshop 1 ⋆ ⋆ ⋆ *Prove the lambda case of the cited Theorem in Volume I using the equation for free variables of a binder.*

Assume $\Gamma, x : A \vdash t : B$ and inductively that free variables of a typed term occur in the context. Since $\mathrm{FV}(\lambda x.t) = \mathrm{FV}(t) \setminus \{x\}$ and the body is typed in $\Gamma, x : A$, every free variable of the abstraction lies in Γ. This discharges the lambda case.

Proof Workshop 2 ⋆ ⋆ ⋆ *Show that alpha-equivalence preserves the set of free variables.*

Alpha-renaming replaces a binder and exactly the occurrences bound by it with a fresh name. Free occurrences are untouched, and freshness prevents a free occurrence from becoming bound. Therefore the free-variable set is invariant; induction on term structure formalizes the argument.

Design Clinic

Design Clinic 1 ⋆ ⋆ ⋆⋆ *Represent the same term once with names and once with de Bruijn indices. Compare which representation makes alpha-equivalence trivial and which makes error messages easier.*

Named form: $\lambda x.\lambda y.x$. With de Bruijn indices where index 0 denotes the nearest binder, the term is $\lambda.\lambda.1$. Indices make alpha-equivalence syntactic equality, but human diagnostics are worse because an index says lexical distance rather than a meaningful source name.

Challenge

Challenge 1 ⋆ ⋆ ⋆ ⋆ ⋆ *Research or derive the de Bruijn representation of $\lambda x.\lambda y.x$. Explain what problem the representation solves and what readability cost it introduces.*

Using the nearest-binder-is-zero convention, $\lambda x.\lambda y.x$ becomes $\lambda.\lambda.1$. The representation solves accidental dependence on binder names and makes alpha-equivalence trivial. Its cost is that insertion/removal of binders shifts indices and raw terms are substantially less readable without a name-restoration layer.

Chapter 5

Substitution: Information in Motion

Checkpoint

Checkpoint 1 ⋆ *Compute* $(x + x)[3/x]$.

Replacing free occurrences of x by 3 gives $3 + 3$.

Checkpoint 2 ⋆ *Explain why* $(\lambda y.x)[y/x]$ *cannot be implemented as naive replacement.*

Naive replacement would produce $\lambda y.y$, where the inserted free y has become bound. Capture-avoiding substitution first alpha-renames the binder to a fresh name, producing for example $\lambda z.y$.

Checkpoint 3 ⋆ *State beta reduction in symbols and in words.*

Symbolically, $(\lambda x.t)\,u \to t[u/x]$ (under the chosen evaluation conditions). In words: applying a lambda supplies the argument for its hypothetical input and propagates that information through the body without changing binding structure.

Core

Core 1 ⋆⋆ *Perform capture-avoiding substitution for* $(\lambda y.\lambda z.x\ y)[y/x]$.

Rename the conflicting outer binder, say y to w: $\lambda w.\lambda z.x\,w$. Then substitute the free x by y, yielding $\lambda w.\lambda z.y\,w$. The inserted y remains free.

Core 2 ⋆⋆ *Explain how the substitution lemma supports preservation.*

The beta case of preservation starts from a derivation of $(\lambda x.t)\,a : B$. Inversion gives $\Gamma, x : A \vdash t : B$ and $\Gamma \vdash a : A$. The substitution lemma supplies $\Gamma \vdash t[a/x] : B$, exactly the type of the reduct.

Core 3 ⋆⋆ *Give an example of substitution into a type family such as* $\mathrm{Vec}(A, n)$.

For a dependent family, substitution acts in types as well as terms: $\mathrm{Vec}(A, n)[3/n] = \mathrm{Vec}(A, 3)$. Volume I previews this operation; Volume II makes it part of typing itself.

Synthesis

Synthesis 1 ⋆ ⋆ ⋆ *Compare beta reduction with function inlining in a compiler. Which properties are shared? Which are not?*

Both beta reduction and inlining replace a formal parameter by an actual argument and can expose further simplifications. Compiler inlining is an optimization on a richer operational language and must preserve effects, evaluation order, code size, calling conventions, and other concerns; beta reduction is a defining computation rule of the calculus.

Synthesis 2 ⋆ ⋆ ⋆ *Describe a database prepared statement as an open term awaiting substitution.*

A prepared statement such as `SELECT ... WHERE id = ?` is an open expression with a typed parameter slot. Binding a value specializes the statement, much as substitution closes an open term. Real databases add effects, serialization, security, and query planning, so the analogy is structural rather than complete.

Proof Workshop

Proof Workshop 1 ⋆ ⋆ ⋆ *Complete the abstraction case of the cited Theorem in Volume I, explicitly stating the freshness condition and where alpha-renaming is used.*

For $\lambda y.u$, if $y = x$, substitution stops under the binder. If $y \neq x$ and $y \notin \mathrm{FV}(a)$, apply the induction hypothesis to u. If $y \in \mathrm{FV}(a)$, alpha-rename y to a fresh z before using the induction hypothesis. Freshness ensures no free variable of a is captured.

Proof Workshop 2 ⋆ ⋆ ⋆ *Prove the variable cases* $x[a/x] = a$ *and* $y[a/x] = y$ *for* $y \neq x$ *preserve typing under the theorem's hypotheses.*

For the matching variable, $x[a/x] = a$, and the theorem hypothesis already gives $\Gamma \vdash a : A$; weakening inserts any trailing independent declarations required by the generalized theorem. For $y \neq x$, $y[a/x] = y$; its declaration is preserved in the target context, so the variable rule re-establishes the same type.

Design Clinic

Design Clinic 1 ⋆ ⋆ ⋆⋆ *Instrument `subst` to emit a trace whenever it freshens a binder. Create the smallest term that triggers the trace and explain why freshening is semantically necessary.*

Use $(\lambda y.x)[y/x]$. An instrumented substitution should report that binder y is freshened, e.g. to $y1$, before replacement, and return $\lambda y1.y$. The trace witnesses the semantic reason for freshening: otherwise the free variable supplied as data would be captured.

Challenge

Challenge 1 ⋆ ⋆ ⋆ ⋆ ⋆ *Sketch an induction proof of substitution for variables, abstraction, and application in simply typed lambda calculus.*

Induct on the typing derivation of the term being substituted into. The variable cases are immediate; application follows by applying the induction hypotheses to function and argument and reusing application typing; abstraction requires the freshness/alpha-renaming case above. The generalized statement with a trailing context is what makes the binder induction hypothesis strong enough.

Chapter 6

Rules: How a Language Grows

Checkpoint

Checkpoint 1 ⋆ *What are the introduction forms for product, function, unit, and empty types?*

Product: $\langle a, b\rangle$. Function: $\lambda x.t$. Unit: $\star$. Empty: none–there is no canonical constructor for $\mathbf{0}$.

Checkpoint 2 ⋆ *What is meant by "elimination after introduction"?*

It means an eliminator is applied directly to data just built by the corresponding introduction rule, such as $\pi_1\langle a, b\rangle$ or $(\lambda x.t)\,u$. A computation rule says how that detour simplifies.

Checkpoint 3 ⋆ *Why is there no introduction rule for* $\mathbf{0}$*?*

An introduction rule for $\mathbf{0}$ would construct evidence of falsehood. Since empty elimination permits an inhabitant of $\mathbf{0}$ to produce any target type, such a constructor would trivialize the logic.

Core

Core 1 ⋆⋆ *Propose computation rules for booleans with an* $\mathtt{if}$ *eliminator.*

Use the standard equations $\mathsf{if}(\mathsf{true}; t; u) \to t$ and $\mathsf{if}(\mathsf{false}; t; u) \to u$, with both branches checked at the same result type.

Core 2 ⋆⋆ *Explain the product eta law in plain language.*

Product eta says a product value contains no observable information beyond its two projections: $p \equiv \langle \pi_1 p, \pi_2 p \rangle$. Repacking everything observable yields the original product up to the chosen equality notion.

Core 3 ⋆⋆ *Give an example of a type former whose eliminator would be too strong and explain the resulting problem.*

Suppose Unit had an eliminator $\frac{\Gamma \vdash u : \mathbf{1}}{\Gamma \vdash \mathsf{magic}_C(u) : C}$ for arbitrary C. Since $\star : \mathbf{1}$, this would construct every type C, including $\mathbf{0}$. The eliminator extracts information never supplied by Unit's sole constructor.

Synthesis

Synthesis 1 ⋆ ⋆ ⋆ *Analyze a file handle type in terms of introduction and elimination. Which operations should be legal? What resource rules might be needed?*

A file handle is introduced by an operation such as `open` and eliminated/used by operations such as `read`, `write`, or `close`. Ordinary weakening/contraction are suspect: silently discarding an open handle may leak a resource, while duplicating it may violate ownership. Linear or affine resource rules can express the intended lifecycle.

Synthesis 2 ⋆ ⋆ ⋆ *Compare an inference-rule constitution with an API specification. What does each omit that the other usually includes?*

Both specify admissible uses compositionally. Inference rules usually omit operational concerns such as latency, effects, failure modes, and representation; API specifications usually omit proof-theoretic properties such as normalization, inversion, or admissibility of structural rules. A rigorous language design often needs both layers.

Proof Workshop

Proof Workshop 1 ⋆ ⋆ ⋆ *For products, derive both beta laws and the eta law from the formation/introduction/elimination/computation template. Mark which are reduction rules and which may instead be definitional equalities.*

Beta laws: $\pi_1\langle a, b\rangle \to a$ and $\pi_2\langle a, b\rangle \to b$ are computation rules. Eta: $p \equiv \langle \pi_1 p, \pi_2 p\rangle$ expresses completeness of observation; a calculus may orient it as reduction/expansion or keep it as definitional/propositional equality to preserve termination properties.

Proof Workshop 2 ⋆ ⋆ ⋆ *Give a type former whose proposed eliminator extracts information not supplied by any constructor. Show the resulting collapse with a concrete derivation.*

The Unit counterexample suffices. From $\star : \mathbf{1}$ and the overpowered eliminator derive a term of any C. Taking $C = \mathbf{0}$ gives inconsistency; taking arbitrary program interfaces gives inhabitants without providing their required information. The rule violates harmony because elimination reveals more than introduction stored.

Design Clinic

Design Clinic 1 ⋆ ⋆ ⋆⋆ *Add a* Maybe(A) *surface form and elaborate it to* $A + \mathbf{1}$. *Specify which layer–surface language, elaborator, or kernel–needs to know the new notation.*

Define surface syntax Maybe(A) and elaborate it to $A + \mathbf{1}$. The parser/elaborator must know the surface notation; the small kernel need only know sum and unit types. Constructors such as `Some a` and `None` can elaborate to $\mathsf{inl}(a)$ and $\mathsf{inr}(\star)$.

Challenge

Challenge 1 ⋆ ⋆ ⋆ ⋆ ⋆ *Investigate the phrase "proof-theoretic harmony." Explain why beta and eta principles can be viewed as two directions of harmony.*

Proof-theoretic harmony says introduction and elimination rules fit one another: elimination should recover no more and no less than the information introduced. Beta principles show eliminators undo fresh constructors; eta principles express that rebuilding from all legitimate observations loses nothing. Excess elimination is unsound; insufficient elimination leaves introduced information unusable.

Chapter 7

Functions: Packaging Hypothetical Computation

Checkpoint

Checkpoint 1 ⋆ *Derive the type of* $\lambda x.x$.

Assume $x : A$. The variable rule gives $x : A$, and arrow introduction discharges the assumption: $\vdash \lambda x.x : A \to A$.

Checkpoint 2 ⋆ *Reduce* $(\lambda x.\mathrm{succ}(x))\ 0$.

Beta reduction substitutes 0 for x: $(\lambda x.\mathrm{succ}(x))\,0 \to \mathrm{succ}(0)$.

Checkpoint 3 ⋆ *Explain function eta in words.*

Function eta says a function is determined by what it does to an argument: a function f is equivalent, under an extensional/eta-aware equality, to $\lambda x.f\,x$ when x is fresh for f.

Core

Core 1 ⋆⋆ *Find a term of type* $A \to B \to A$.

$\lambda x.\lambda y.x$.

Core 2 ⋆⋆ *Find a term of type* $(A \to B) \to (B \to C) \to A \to C$.

$\lambda f.\lambda g.\lambda x.g(f\,x)$.

Core 3 ⋆⋆ *Explain why the rules introduced so far provide no closed* constructor *for an arbitrary atomic type A. What additional metatheoretic fact is needed to conclude that no closed inhabitant exists at all?*

No introduction rule concludes an arbitrary atomic A, so there is no closed canonical constructor of A. To infer that there is no closed inhabitant at all, one needs normalization (or an equivalent logical-relations/canonical-model argument) so any hypothetical closed inhabitant would reduce to a canonical form, which cannot exist.

Synthesis

Synthesis 1 ⋆ ⋆ ⋆ *Read the type* $(A \to B) \to A \to B$ *as an English protocol.*

First supply a transformer $f : A \to B$. The result is then a capability that waits for an A; after an A arrives, it applies f and returns a B. The type describes a two-stage information protocol.

Synthesis 2 ⋆ ⋆ ⋆ *Compare partial application with a conversation in which information arrives in stages.*

Partial application records information already received and returns a computation waiting for the remainder. A conversation behaves similarly: after the first message, the set or meaning of valid continuations can change. Simple arrows capture fixed staged input; dependent types later allow the next type itself to change with earlier content.

Proof Workshop

Proof Workshop 1 ⋆ ⋆ ⋆ *Give a complete typing derivation for the composition term in Worked Example 7.1.*

In context $f : A \to B, g : B \to C, x : A$, application gives $fx : B$, then $g(fx) : C$. Discharge x, then g, then f by three arrow-introduction steps to obtain $(A \to B) \to (B \to C) \to A \to C$.

Proof Workshop 2 ⋆ ⋆ ⋆ *Use the cited Theorem in Volume I to show that applying the identity function to any well-typed argument preserves its type after one beta step.*

From $\Gamma \vdash a : A$, the identity has $\Gamma \vdash \lambda x.x : A \to A$. Application yields $\Gamma \vdash (\lambda x.x)\, a : A$, and beta reduction gives a. The beta-typing stability result (ultimately substitution) therefore preserves type A in the step.

Design Clinic

Design Clinic 1 ⋆ ⋆ ⋆⋆ *Compare two interfaces for lambdas: annotated binders versus bidirectional checking against an expected arrow type. Identify what information each interface asks the programmer to provide.*

Annotated binders ask the programmer to state the input type locally, so a lambda can often synthesize an arrow. Bidirectional checking instead receives an expected $A \to B$, enters $x : A$, and checks the body at B; the programmer can omit the binder annotation but the surrounding context must provide the expected type. The tradeoff is local verbosity versus contextual information flow.

Challenge

Challenge 1 ⋆ ⋆ ⋆ ⋆ ⋆ *Derive a term witnessing* $((A \to B) \to A) \to A$ *if classical control operators are admitted. Explain why the pure intuitionistic lambda calculus does not give this inhabitant for arbitrary* A, B.

The type is Peirce's law. With an answer-type-polymorphic control operator having scheme $\mathsf{callcc} : ((A \to B) \to A) \to A$, the witness is simply $\lambda f.\mathsf{callcc}(f)$ (or the primitive applied directly, depending on syntax). Pure intuitionistic lambda calculus has no uniform inhabitant because Peirce's law is not intuitionistically valid; adding classical continuations or a classical axiom changes the logic and operational semantics.

Chapter 8

Products and Sums: The Algebra of Information

Checkpoint

Checkpoint 1 ⋆ *Give the canonical introduction form of $A \times B$.*

$\langle a, b \rangle$ with $a : A$ and $b : B$.

Checkpoint 2 ⋆ *Why must case analysis for $A + B$ have two branches?*

A value of $A + B$ can have been introduced by either inl or inr. To define a total consumer without inspecting impossible hidden data, elimination must cover both constructor cases.

Checkpoint 3 ⋆ *Encode a boolean using unit and sum.*

Bool $\cong \mathbf{1} + \mathbf{1}$: represent false by $\mathsf{inl}(\star)$ and true by $\mathsf{inr}(\star)$ (or vice versa).

Core

Core 1 ⋆⋆ *Write programs witnessing $A \times B \cong B \times A$.*

Forward $\lambda p.\langle \pi_2 p, \pi_1 p \rangle$; backward is the same shape. On a canonical pair, composing either direction beta/projection-reduces to the original pair, up to product eta if needed.

Core 2 ⋆⋆ *Write programs witnessing $A + B \cong B + A$.*

Forward $\lambda s.\mathsf{case}(s; x.\mathsf{inr}(x); y.\mathsf{inl}(y))$; backward swaps tags in the same way. Each composite restores the original tag and payload.

Core 3 ⋆⋆ *Show that $A \times \mathbf{1} \cong A$ and $A + \mathbf{0} \cong A$.*

For $A \times \mathbf{1} \cong A$, use $p \mapsto \pi_1 p$ and $a \mapsto \langle a, \star \rangle$. For $A + \mathbf{0} \cong A$, eliminate the left injection by identity and the impossible right injection by empty elimination; the inverse is $a \mapsto \mathsf{inl}(a)$.

Synthesis

Synthesis 1 ⋆ ⋆ ⋆ *Design the type of a parser that may either return an AST or structured parse error information.*

A simple type is $\mathsf{Input} \to (\mathsf{AST} + \mathsf{ParseError})$. A richer parser might return a product containing source span, recovery information, or warnings. The sum makes success/failure explicit and forces callers to handle both cases.

Synthesis 2 ⋆ ⋆ ⋆ *Explain how exhaustive pattern matching is a direct computational consequence of the introduction rules for sums.*

The constructors of a sum establish that every canonical value is either left or right. A total eliminator must therefore provide one continuation for each introduction possibility. Exhaustiveness checking is the algorithmic reflection of this canonical-forms fact.

Proof Workshop

Proof Workshop 1 ⋆ ⋆ ⋆ *Prove the product clause of the cited Theorem in Volume I by enumerating every TinyTT value form and excluding the impossible cases by typing inversion.*

TinyTT values are lambdas, unit, booleans, zero/successors, pairs, and injections. If a closed value has type $A \times B$, inversion excludes every value constructor except Pair; a Pair's typing rule then gives its two component types. Thus a closed value of product type must be $\langle v_1, v_2 \rangle$ with $v_1 : A$ and $v_2 : B$.

Proof Workshop 2 ⋆ ⋆ ⋆ *Construct programs witnessing both directions of $A \times (B + C) \cong (A \times B) + (A \times C)$, then normalize both composites on canonical inputs.*

Forward: $\lambda p.\mathsf{case}(\pi_2 p; b.\mathsf{inl}\langle \pi_1 p, b \rangle; c.\mathsf{inr}\langle \pi_1 p, c \rangle)$. Backward: case on the outer sum, mapping $\langle a, b \rangle$ to $\langle a, \mathsf{inl}(b) \rangle$ and $\langle a, c \rangle$ to $\langle a, \mathsf{inr}(c) \rangle$. On either canonical tag, the composite reduces by case and projection beta rules to the starting value.

Design Clinic

Design Clinic 1 ⋆ ⋆ ⋆⋆ *Encode a small JSON-like message schema as a sum of products. Explain which malformed payload shapes become unrepresentable after checking.*

For example, Message = Login(User × Password) + Ping(**1**) + Data(Id × Bytes) (nested binary sums in the core). Once checked, a Login cannot omit its password, Ping cannot carry arbitrary bytes, and Data cannot substitute a password-shaped payload unless the field types agree.

Challenge

Challenge 1 ⋆ ⋆ ⋆ ⋆ ⋆ *Give explicit functions in both directions for the distributive isomorphism $A \times (B + C) \cong (A \times B) + (A \times C)$ and reason informally that the composites reduce to identities.*

Use the same two functions as in Proof Workshop 2. For an input $\langle a, \mathsf{inl}(b)\rangle$, forward yields $\mathsf{inl}\langle a, b\rangle$ and backward returns $\langle a, \mathsf{inl}(b)\rangle$; the right case is symmetric. Conversely, starting from either outer injection, backward then forward restores that injection. Thus the composites are identities on canonical values, extending by the chosen equality principle.

Chapter 9

Natural Numbers and Induction: Computation from Generation

Checkpoint

Checkpoint 1 ⋆ *State the canonical constructors of* $\mathbb{N}$.

$0 : \mathbb{N}$ and, from $n : \mathbb{N}$, $\mathrm{succ}(n) : \mathbb{N}$.

Checkpoint 2 ⋆ *State the two computation equations of the natural-number recursor.*

$\mathrm{rec}_{\mathbb{N}}(0; z; s) \to z$ and $\mathrm{rec}_{\mathbb{N}}(\mathrm{succ}(n); z; s) \to s(\mathrm{rec}_{\mathbb{N}}(n; z; s))$ in the reduction-generated equality (with evaluation order handled by the operational semantics).

Checkpoint 3 ⋆ *Explain the difference between recursion and dependent induction.*

Recursion returns data in a fixed result type C. Dependent induction returns evidence in a family $P(n)$ whose target type changes with the number being analyzed; the step transports evidence from $P(n)$ to $P(n+1)$.

Core

Core 1 ⋆⋆ *Define predecessor using natural-number recursion, choosing a suitable state type if needed.*

Use state $C = \mathbb{N} \times \mathbb{N}$. Start at $\langle 0, 0\rangle$ and step $\langle k, p\rangle \mapsto \langle \mathrm{succ}(k), k\rangle$. Then $\mathrm{pred}(n) = \pi_2(\mathrm{rec}_{\mathbb{N}}(n; \langle 0, 0\rangle; \mathrm{step}))$. It returns 0 at zero and the predecessor thereafter.

Core 2 ⋆⋆ *Define a boolean test for zero.*

$\mathrm{isZero}(n) = \mathrm{rec}_{\mathbb{N}}(n; \mathrm{true}; \lambda\_.\mathrm{false})$. The first successor discards the accumulated result and returns false; further successors remain false.

Core 3 ⋆⋆ *Explain why structural recursion on naturals terminates.*

On a canonical numeral, every recursive call is made on the immediate predecessor, so constructor depth strictly decreases until zero. This establishes termination of the recursor on numerals; a global strong-normalization theorem is a larger metatheoretic statement.

Synthesis

Synthesis 1 ⋆ ⋆ ⋆ *Describe the constructors and eliminator shape for a binary tree type.*

For a binary tree, constructors might be $\mathsf{leaf} : A \to \mathsf{Tree}(A)$ and $\mathsf{node} : \mathsf{Tree}(A) \times \mathsf{Tree}(A) \to \mathsf{Tree}(A)$. The eliminator supplies a leaf case and a node case receiving recursive results for both subtrees.

Synthesis 2 ⋆ ⋆ ⋆ *Explain how an AST interpreter is structurally similar to a proof by induction on syntax.*

An AST is generated by finitely many syntax constructors. An interpreter follows those constructors recursively; an induction proof follows the same cases but carries a proposition instead of an output value. Both are folds over construction history.

Proof Workshop

Proof Workshop 1 ⋆ ⋆ ⋆ *Expand the $n = 2$ case of the cited Theorem in Volume I as a sequence of recursor equations and congruence/definitional-equality steps. Explain why, when the step and base are open terms, this need not be a literal call-by-value execution trace.*

For $\overline{2} = \mathsf{succ}(\mathsf{succ}(0))$, use the recursor equations: $\mathsf{rec}(\overline{2}; z; s) \equiv s(\mathsf{rec}(\overline{1}; z; s)) \equiv s(s(\mathsf{rec}(0; z; s))) \equiv s(s(z))$. The equalities use computation plus congruence. If s or z are open, a call-by-value machine need not literally traverse the printed sequence because open variables are not executable values in TinyTT.

Proof Workshop 2 ⋆ ⋆ ⋆ *Define multiplication using the recursor and prove by induction on the first input that* $\mathsf{mul}\ \overline{n}\ \overline{m}$ *reduces to the numeral for nm, assuming the corresponding theorem for addition.*

Define $\mathsf{mul}(n, m) = \mathsf{rec}(n; 0; \lambda acc.\mathsf{add}(m, acc))$. Base $n = 0$: result $0 = 0 \cdot m$. Step: assuming the recursive result reduces to nm, one recursor equation gives $\mathsf{add}(m, nm)$, which by the addition theorem reduces to $(n + 1)m$. This is induction on the first numeral.

Design Clinic

Design Clinic 1 ⋆ ⋆ ⋆⋆ *Add a predecessor function to TinyTT without adding a primitive predecessor constructor. Derive it from natural-number recursion and test it on zero and three.*

Implement exactly the pair-state construction from Core 1, annotating the zero-case pair at $\mathbb{N} \times \mathbb{N}$ if the bidirectional checker needs it. Tests: $\mathsf{pred}(0) \Downarrow 0$; three steps from $\langle 0, 0 \rangle$ yield $\langle 3, 2 \rangle$, so $\mathsf{pred}(3) \Downarrow 2$.

Challenge

Challenge 1 ⋆ ⋆ ⋆ ⋆ ⋆ *Formulate a dependent predicate $P(n)$ expressing "$n + 0 = n$" and describe the base and induction-step inhabitants that a full dependent type theory would require.*

Take $P(n) = \mathsf{Id}_{\mathbb{N}}(\mathsf{add}(n, 0), n)$ in a theory with identity types. Base evidence proves $\mathsf{add}(0, 0) = 0$. The step assumes $p : P(n)$ and uses congruence of successor, together with the computation equation for addition, to construct $P(\mathsf{succ}(n))$. The crucial feature is that the evidence type changes with n.

Chapter 10

Equality and Conversion: When Are Two Programs the Same?

Checkpoint

Checkpoint 1 ⋆ *Give three distinct meanings of "program equality."*

Examples: syntactic identity (same text/tree), alpha-equivalence (same binding modulo bound names), definitional equality (same by computation/congruence), propositional equality (an inhabitant of an identity/equality type), extensional equality (same observable mapping), and contextual equivalence (no program context can distinguish them).

Checkpoint 2 ⋆ *Why should* $(\lambda x.x)\ 0$ *and* 0 *normally be definitionally equal?*

The application immediately exposes the lambda's body with 0 substituted for x, producing 0. Treating the redex and reduct as definitionally equal makes computation transparent to typing and avoids requiring explicit proofs for routine execution.

Checkpoint 3 ⋆ *State the conversion principle in words.*

If $\Gamma \vdash t : A$ and $A \equiv B$ type, conversion permits $\Gamma \vdash t : B$. In words, the checker may change a term's assigned type along an equality that the theory treats as definitional.

Core

Core 1 ⋆⋆ *Draw a confluence diamond for a term containing two independent beta redexes.*

Use $\langle (\lambda x.x)0, (\lambda y.y)\mathsf{true}\rangle$. Reducing the left redex first or the right redex first produces two intermediate terms, and reducing the remaining redex makes both paths meet at $\langle 0, \mathsf{true}\rangle$.

Core 2 ⋆⋆ *Give two extensionally equal functions that might not be syntactically identical.*

On naturals, compare $f = \lambda n.n$ with $g = \lambda n.\mathrm{rec}_{\mathbb{N}}(n; 0; \lambda k.\mathrm{succ}(k))$. For every canonical numeral they compute to the same numeral, yet with an open variable n the recursor may remain stuck syntactically, so the functions need not be definitionally identical in a weak reduction-generated equality.

Core 3 ⋆⋆ *Explain why unrestricted theorem proving inside definitional equality threatens decidable type checking.*

If definitional equality invokes unrestricted theorem proving, type checking inherits the theorem prover's potential nontermination or undecidability. A programmer could no longer rely on the checker to terminate mechanically on every input.

Synthesis

Synthesis 1 ⋆ ⋆ ⋆ *Compare definitional equality with compiler constant folding.*

Both evaluate known structure at compile/check time: $(\lambda x.x)0$ and arithmetic constants can be simplified before run time. Definitional equality is part of the language's formal notion of sameness and may be required for type checking; constant folding is an optimization that need only preserve semantics.

Synthesis 2 ⋆ ⋆ ⋆ *Explain why equality policy is part of language ergonomics, not merely foundations.*

Equality policy determines which routine transformations happen silently, which require annotations or proof terms, and whether error messages compare normalized or surface forms. It therefore directly controls convenience, predictability, checker cost, and what programmers must state explicitly.

Proof Workshop

Proof Workshop 1 ⋆ ⋆ ⋆ *Identify exactly where confluence and strong normalization are used in the cited Theorem in Volume I. Give a failure mode if each assumption is removed.*

Strong normalization ensures normalization terminates; without it, the normalization procedure may diverge. Confluence ensures the normal form is independent of reduction choices; without it, two reduction paths may reach distinct irreducible forms, so comparing arbitrary chosen normal forms is unsound as a decision procedure for the intended equality.

Proof Workshop 2 ⋆ ⋆ ⋆ *Show that alpha-equivalence is an equivalence relation on the named lambda terms used in TinyTT.*

Reflexivity uses the identity renaming. Symmetry inverts the bijective correspondence of bound names. Transitivity composes consistent fresh renamings/binding correspondences. A structural induction, preferably phrased with a binding environment or de Bruijn representation, handles the binder case without capture.

Design Clinic

Design Clinic 1 ⋆⋆⋆⋆ *Design an "equality budget" for a language: list three equalities you would make automatic and two you would require explicit proof for. Justify the checker-cost tradeoff.*

A defensible budget might automate alpha-equivalence, beta/projection/case/recursor computation, and a terminating eta policy for selected simple types; require explicit proofs for arbitrary arithmetic theorems and semantic/contextual equivalence. The automatic layer is local and decidable; the explicit layer can be more expressive without infecting kernel conversion with unbounded search.

Challenge

Challenge 1 ⋆ ⋆ ⋆ ⋆ ⋆ *Investigate normalization-by-evaluation at a conceptual level. Explain why evaluating into a semantic domain and reifying back to syntax can decide equality without performing naive source-level rewriting.*

Normalization-by-evaluation maps syntax into a semantic domain where functions are represented extensionally enough to evaluate open structure, then reifies semantic values to canonical syntax. Equality is decided by comparing reified normal forms. It avoids repeatedly searching source terms for redexes and often yields a structurally recursive normalization algorithm; correctness still requires a soundness/completeness argument for the semantic interpretation.

Chapter 11

Normalization: Computation as the Exposure of Canonical Form

Checkpoint

Checkpoint 1 ⋆ *Distinguish value, normal form, and canonical form.*

A value is an operationally finished result under a chosen evaluation strategy. A normal form has no reduction step anywhere relevant to the reduction relation. A canonical form is an introduction-shaped value appropriate to a type, such as a lambda at arrow type or pair at product type. These notions can coincide in simple settings but are conceptually distinct.

Checkpoint 2 ⋆ *State progress and preservation.*

Progress: a closed well-typed term is a value or can step. Preservation: if a well-typed term steps, the reduct has the same type (under the stated context).

Checkpoint 3 ⋆ *Give the untyped diverging term Ω and explain its behavior.*

$\Omega = (\lambda x.x\,x)(\lambda x.x\,x)$. Beta contraction reproduces the same term, so reduction loops forever.

Core

Core 1 ⋆⋆ *Give a term on which call-by-value performs work that call-by-name avoids.*

$(\lambda x.0)\,\Omega$ is the classic example. Call-by-name can beta-reduce the outer application to 0 without evaluating the unused argument; call-by-value attempts to evaluate Ω and diverges. The example is intentionally untyped in the simply typed core.

Core 2 ⋆⋆ *Explain why preservation's beta case requires substitution.*

In the beta case, the reduct is obtained by substitution. Inversion of typing gives a body typed under $x : A$ and an argument typed at A; only the substitution theorem converts those premises into a typing derivation for the substituted body.

Core 3 ⋆⋆ *State a canonical-forms lemma for products.*

If a closed value has type $A \times B$, it must be a pair $\langle v_1, v_2 \rangle$ with $v_1 : A$ and $v_2 : B$. This follows by enumerating the value grammar and eliminating incompatible outer constructors by typing inversion.

Synthesis

Synthesis 1 ⋆ ⋆ ⋆ *Explain why a server may be correct precisely because it does not terminate.*

A server's specification is often to remain available and repeatedly react to requests. Termination would mean the service has stopped. Correctness is therefore a property of traces, responsiveness, invariants, and permitted interactions rather than convergence to one final value.

Synthesis 2 ⋆ ⋆ ⋆ *Distinguish type safety from memory safety, total correctness, and semantic correctness.*

Type safety rules out specified stuck states according to the language's statics/dynamics. Memory safety concerns invalid memory access; total correctness adds termination and postconditions; semantic correctness means satisfying the intended application specification. One property can hold without the others.

Proof Workshop

Proof Workshop 1 ⋆ ⋆ ⋆ *Complete one congruence case and one computational case in the proof of preservation, showing the typing derivation before and after the step.*

Congruence example: if $t \to t'$ and $\Gamma \vdash t : A$, then $\mathrm{succ}(t) \to \mathrm{succ}(t')$ and preservation of the premise gives $t' : A$, so successor typing re-applies. Computational example: $\pi_1\langle a, b \rangle \to a$; inversion of pair typing already yields $a : A$, the result type of the projection.

Proof Workshop 2 ⋆ ⋆ ⋆ *Prove progress for closed applications from the induction hypotheses and the canonical-form fact for arrow values.*

From $\vdash f\,a : B$, inversion gives $f : A \to B$ and $a : A$. By induction, either f steps or is a value. If it steps, lift the step. If it is a value, canonical forms makes it a lambda. Then either a steps (lift that step) or is a value, in which case beta reduction applies. Thus the application is never a closed stuck non-value.

Design Clinic

Design Clinic 1 ⋆ ⋆ ⋆⋆ *Modify the evaluator to return a structured reason when `step` returns `None`: value, impossible stuck state, or unsupported form. Explain how progress predicts which category closed well-typed terms can occupy.*

Return an enum or tagged record such as `Value`, `StuckInvariantViolation`, and `UnsupportedEvaluatorForm`, with the term and source rule. For a closed term already accepted by the declarative typing discipline, progress predicts that only `Value` or an available step can occur; reaching `StuckInvariantViolation` is evidence of an implementation or theorem-assumption mismatch.

Challenge

Challenge 1 ⋆ ⋆ ⋆ ⋆ ⋆ *Sketch the progress proof for application: if $t\,u$ is closed and well typed, analyze the possible states of t and u until a reduction rule or value case is forced.*

The proof is the same case split as Proof Workshop 2, extended to evaluation order: first force the function position; canonical forms makes a function value a lambda; then force the argument as required by call-by-value; when both are values, beta applies. The key ingredients are induction hypotheses plus the arrow canonical-forms lemma, not normalization.

Chapter 12

Proof and Program: The Curry-Howard Lens

Checkpoint

Checkpoint 1 ⋆ *Match implication, conjunction, disjunction, truth, and falsehood to their type constructors.*

Implication $A \Rightarrow B \leftrightarrow A \to B$; conjunction $A \wedge B \leftrightarrow A \times B$; disjunction $A \vee B \leftrightarrow A + B$; truth $\top \leftrightarrow \mathbf{1}$; falsehood $\bot \leftrightarrow \mathbf{0}$.

Checkpoint 2 ⋆ *Why is constructive evidence for a disjunction computationally informative?*

A constructive proof of $A \vee B$ must identify which side holds and supply evidence for that side. Computationally it therefore contains a tag plus payload, exactly the information represented by a sum value.

Checkpoint 3 ⋆ *Define negation using the empty type.*

$\neg A$ is represented as $A \to \mathbf{0}$: evidence of A would lead to an impossibility.

Core

Core 1 ⋆⋆ *Find inhabitants of* $(A \times B) \to A$ *and* $(A \times B) \to B$.

$\lambda p.\pi_1 p : (A \times B) \to A$ and $\lambda p.\pi_2 p : (A \times B) \to B$.

Core 2 ⋆⋆ *Find an inhabitant of* $A \to (B \to A \times B)$.

$\lambda a.\lambda b.\langle a, b\rangle$.

Core 3 ⋆⋆ *Explain why an existential proposition naturally corresponds to a dependent pair rather than an ordinary product.*

Constructive existence must carry both a witness $x : A$ and evidence whose type is $B(x)$. Because the second component's type depends on the first component's value, the correct structure is $\Sigma_{x:A} B(x)$ rather than a fixed product $A \times B$.

Synthesis

Synthesis 1 ⋆ ⋆ ⋆ *Explain program extraction from a constructive proof to a software engineer.*

Under Curry–Howard, a constructive proof is a typed construction. Erasing proof-irrelevant parts and retaining computational witnesses can therefore yield an executable program whose type/specification was justified by the proof. The extraction theorem must state which logical features are computationally interpreted and which are erased.

Synthesis 2 ⋆ ⋆ ⋆ *Explain proof erasure to a mathematician concerned that proofs will make programs inefficient.*

Proof erasure removes terms that affect only static justification when the theory guarantees they cannot affect runtime behavior. The resulting program need not carry every proof object. Some proofs are computationally relevant, however–for example a constructive existential carries its witness–so erasure is a typed transformation, not deletion by convention.

Proof Workshop

Proof Workshop 1 ⋆ ⋆ ⋆ *Translate the proof of $(A \wedge B) \Rightarrow (B \wedge A)$ into a typing derivation and then into a TinyTT term.*

Assume $p : A \wedge B$. Conjunction elimination yields $\pi_2 p : B$ and $\pi_1 p : A$; conjunction introduction gives $\langle \pi_2 p, \pi_1 p \rangle : B \wedge A$. The corresponding term is $\lambda p.\langle \pi_2 p, \pi_1 p \rangle : (A \times B) \to (B \times A)$.

Proof Workshop 2 ⋆ ⋆ ⋆ *Exhibit an implication-introduction/elimination detour and perform the cited Proposition in Volume I at both the derivation and term levels.*

A detour is: assume $x : A$, derive $t : B$, introduce $\lambda x.t : A \to B$, then immediately eliminate it with $a : A$. Proof normalization substitutes the derivation of a for uses of assumption x in the derivation of t; term normalization is $(\lambda x.t)a \to t[a/x]$.

Design Clinic

Design Clinic 1 ⋆ ⋆ ⋆⋆ *Choose a constructive theorem with nontrivial computational content. Identify which parts of its proof should remain after proof erasure and which parts are logically necessary but operationally irrelevant.*

Example: constructive sorting can return both a list and evidence that it is a permutation and ordered. Runtime data needed downstream (the sorted list, perhaps comparison decisions) remains; purely proof-level certificates may be erased if the target runtime does not inspect them. The rubric is semantic: erase only evidence whose elimination is restricted so it cannot influence retained computation.

Challenge

Challenge 1 ⋆ ⋆ ⋆ ⋆ ⋆ *Determine which classical tautologies fail to have inhabitants in the pure intuitionistic fragment and identify what additional control or logical principle would be required.*

Examples include excluded middle $A + (A \to \mathbf{0})$, double-negation elimination $((A \to \mathbf{0}) \to \mathbf{0}) \to A$, and Peirce's law $((A \to B) \to A) \to A$. They lack uniform inhabitants in pure intuitionistic simple type theory. One may add classical axioms as constants or give them computational meaning via continuations/control operators; either choice changes the system's logic or dynamics.

Chapter 13

Type Checking: Turning Judgments into an Algorithm

Checkpoint

Checkpoint 1 ⋆ *Distinguish synthesis from checking.*

Synthesis computes a type from a term and context, written conceptually $\Gamma \vdash t \Rightarrow A$. Checking receives an expected type and verifies the term against it, $\Gamma \vdash t \Leftarrow A$.

Checkpoint 2 ⋆ *Why do lambdas naturally check rather than synthesize in a basic bidirectional system?*

A bare lambda does not contain the domain type of its binder. Given an expected arrow $A \to B$, checking can enter $x : A$ and check the body at B without guessing. Synthesis would need an annotation or inference variables/constraints.

Checkpoint 3 ⋆ *What role does elaboration play between surface syntax and the kernel?*

Elaboration translates rich surface syntax, implicit arguments, notation, and inferred information into a smaller explicit core. The kernel then checks the core object. This separates user convenience from the trusted validity judgment.

Core

Core 1 ⋆⋆ *Write algorithmic rules for pair synthesis or checking.*

A synthesis rule can require $\Gamma \vdash a \Rightarrow A$ and $\Gamma \vdash b \Rightarrow B$ and conclude $\Gamma \vdash \langle a, b \rangle \Rightarrow A \times B$. A more permissive bidirectional design makes pairs checking forms: given expected $A \times B$, check each component at its corresponding type. The latter handles bare lambdas inside pairs more smoothly.

Core 2 ⋆⋆ *Design a type error that reports a failed function application with context.*

For `f a`, report the synthesized type of `f`, the expected domain, the actual/failed type of `a`, and the context entries relevant to both. Preserve a breadcrumb such as "checking pair.second -> checking application.argument" so the user sees the failed rule premise rather than only a final mismatch.

Core 3 ⋆⋆ *Explain why a small kernel can increase trust even if the elaborator is large.*

A small kernel reduces the amount of code whose correctness must be trusted for acceptance. A large elaborator may search, infer, optimize, or even contain bugs; if its output is independently rechecked by a sound kernel, an elaborator bug normally causes rejection or a different core term rather than acceptance of an invalid core judgment. Correct elaboration of user intent remains a separate concern.

Synthesis

Synthesis 1 ⋆ ⋆ ⋆ *Compare proof-assistant architecture with a compiler frontend plus verified intermediate representation.*

The elaborator resembles a compiler frontend that resolves surface conveniences into an explicit intermediate representation; the proof kernel resembles a small verifier for that IR. The analogy is strongest when acceptance depends only on the verifier and the IR has a precise semantics.

Synthesis 2 ⋆ ⋆ ⋆ *Explain how normalization affects the termination of dependent type checking.*

Dependent checking often compares types containing computations. If definitional equality normalizes those computations, checker termination depends on normalization/decidability properties of the type-level language. General unrestricted recursion at the type level can therefore make conversion, and hence checking, nonterminating or undecidable.

Proof Workshop

Proof Workshop 1 ⋆ ⋆ ⋆ *Prove the application branch of the cited Theorem in Volume I directly from the code contract of `synth` and the declarative application rule.*

In the application branch, `synth(f)` returns $A \to B$ and `check(a,A)` succeeds. By mutual induction, these algorithmic successes imply declarative derivations $\Gamma \vdash f : A \to B$ and $\Gamma \vdash a : A$. The declarative application rule then yields $\Gamma \vdash fa : B$, matching the synthesized result.

Proof Workshop 2 ⋆ ⋆ ⋆ *Give a declaratively typable bare lambda that TinyTT cannot synthesize. Add the smallest annotation that restores an algorithmic path.*

$\lambda x.x$ is declaratively typable at $A \to A$ but TinyTT does not synthesize a type for a bare lambda. The smallest repair is an annotation on the whole term, $(\lambda x.x : A \to A)$ in the AST as `Ann(Lam(...), Arrow(A,A))`; synthesis checks the lambda against the annotation and returns the arrow.

Design Clinic

Design Clinic 1 ⋆ ⋆ ⋆⋆ *Add a derivation breadcrumb stack to the checker so a type mismatch reports the nested rule path that led to it. Demonstrate the output on an ill-typed application inside a pair.*

Maintain a stack of frames when descending: e.g. `Pair(second)`, `App(argument)`, `Check(expected=Nat)`. On failure, print the frames outer-to-inner plus the local premise. For a pair whose second component contains `f true` while $f : \mathbb{N} \to \mathbb{N}$, the diagnostic should point to `true`, report expected Nat/got Bool, and show that the obligation arose as the application's argument inside the pair's second field.

Challenge

Challenge 1 ⋆ ⋆ ⋆ ⋆ ⋆ *Give a case where declarative typing is elegant but not syntax directed. Propose an annotation or bidirectional rule that restores a deterministic checking path.*

Subsumption/conversion is a standard declarative example: from $t : A$ and $A \equiv B$, infer $t : B$, but this rule can be applied at many points and is not syntax directed. A bidirectional design concentrates conversion at checking boundaries, or requires an explicit annotation/cast, so the algorithm knows when to compare an inferred type with an expected one.

Chapter 14

Judgment as Communication: From Expressions to Protocol States

Checkpoint

Checkpoint 1 ⋆ *Give a communication reading of* $\Gamma \vdash a : A$.

Read Γ as shared assumptions/capabilities, a as the transmitted construction or action, and A as the contract describing what kind of message/action it is. The judgment says the communication is licensed relative to that shared state. This is a structural reading, not yet a temporal protocol semantics.

Checkpoint 2 ⋆ *Why can* $A \to B$ *be read as a tiny protocol?*

$A \to B$ describes a one-request/one-response capability: provide an A and receive a B. It is "tiny" because the response type is fixed and there is no explicit state transition, concurrency, failure, or multi-round sequencing.

Checkpoint 3 ⋆ *State one computational specification that simple types cannot express naturally but dependent types can.*

A vector whose length appears in its type, $\mathrm{Vec}(A, n)$; a response type selected by the request tag; or evidence whose proposition mentions a computed value. Each needs a type family indexed by term-level information.

Core

Core 1 ⋆⋆ *Reinterpret context extension as a protocol-state transition.*

Context extension $\Gamma \rightsquigarrow \Gamma, x : A$ can be read as learning/receiving a message $x : A$ and making it available for subsequent judgments. The analogy becomes literal only in a

language with an explicit temporal/session semantics; ordinary proof contexts do not themselves record time or message consumption.

Core 2 ⋆⋆ *Explain how an API's type language determines which misuse can be rejected before execution.*

The API's type language determines which distinctions are statically representable: nullability, ownership, request variants, effect capabilities, dimensions, protocol states, and so on. Misuses that correspond to distinctions absent from the type language cannot be rejected by typing alone.

Core 3 ⋆⋆ *Explain the phrase "collected comprehension principles" using three type formers.*

Products add a principle for forming worlds containing two pieces simultaneously; sums add a principle for tagged alternatives; arrows add a principle for packaging a construction valid under a hypothetical input. A type theory can be viewed as a collection of such formation/introduction/elimination/computation principles that determine which structured possibilities are expressible.

Synthesis

Synthesis 1 ⋆ ⋆ ⋆ *Choose an interactive system – database, robot, browser, or agent – and identify which parts fit the closed function model and which require a richer protocol model.*

For a database, a closed function model fits pure query planning from an immutable schema to a plan. Transactions, locks, failures, streaming results, and concurrent updates require richer state/effect/protocol descriptions. A useful analysis separates pure transformations from interactions whose legality depends on history.

Synthesis 2 ⋆ ⋆ ⋆ *Defend or criticize the claim that programming-language design is partly ontology design: choosing which computational distinctions exist in the language.*

The claim is defensible in a precise sense: choosing types chooses which entities and distinctions the language can name and enforce–e.g. option versus nullable sentinel, owned versus borrowed resource, authenticated versus unauthenticated state. It should not be inflated into metaphysics: the language's ontology is engineered and partial, and runtime/physical distinctions may exist outside it.

Proof Workshop

Proof Workshop 1 ⋆ ⋆ ⋆ *Prove the cited Proposition in Volume I by writing the simple arrow formation rule and the dependent product formation rule side by side and isolating the changed context.*

Simple arrow formation is schematically $\frac{\Gamma \vdash A \text{ type} \quad \Gamma \vdash B \text{ type}}{\Gamma \vdash A \to B \text{ type}}$. Dependent product formation changes the second premise to $\Gamma, x : A \vdash B(x)$ type, concluding $\Gamma \vdash \Pi_{x:A} B(x)$ type. The changed context is the entire threshold: the codomain may now mention the particular input.

Proof Workshop 2 ⋆ ⋆ ⋆ *Give three nonconstant families $B(x)$ drawn from data shape, protocol state, and proof evidence. State exactly what information indexes each family.*

Data shape: $B(n) = \mathsf{Vec}(A, n)$ indexed by length $n : \mathbb{N}$. Protocol state: $B(r) = \mathsf{Response}(r)$ indexed by a request/tag $r : \mathsf{Request}$. Proof evidence: $B(n) = \mathsf{Id}_{\mathbb{N}}(\mathsf{sortLength}(n), n)$ or more simply $P(n)$ indexed by the value about which evidence is asserted. In each case the family is genuinely nonconstant.

Design Clinic

Design Clinic 1 ⋆ ⋆ ⋆⋆ *Specify a two-message request/response protocol in which the response type depends on the request tag. Explain why a simple sum of all responses loses information that a dependent response family retains.*

Let $\mathsf{Request} = \mathsf{GetUser}(\mathsf{Id}) + \mathsf{Ping}(\mathbf{1})$. Define $\mathsf{Response}(\mathsf{GetUser}(i)) = \mathsf{User}$ and $\mathsf{Response}(\mathsf{Ping}(\star)) = \mathsf{Pong}$. A simple sum User + Pong permits a Pong after GetUser unless additional checking remembers the request. The dependent family encodes the correlation in the type itself.

Challenge

Challenge 1 ⋆ ⋆ ⋆ ⋆ ⋆ *Give the strongest version of the "type theory is the grammar of computation" thesis that you now consider defensible, and list three empirical or formal counterexamples that a complete theory would still have to answer.*

A strong defensible thesis is: type theory provides a general compositional language for specifying classes of computational constructions, the information they depend on, their admissible observations, and invariants preserved by composition/evaluation. It is not established that all computation is best or uniquely captured this way. Stress cases include untyped/general-recursive computation, open concurrent/interactive systems, stochastic and continuous/physical computation, and learning systems whose relevant state or semantics is not statically enumerable. A genuine unification must model these cases or state principled boundaries.

Chapter A

Difficulty Calibration and Grading Notes

Mode	Nominal	Expected evidence
Checkpoint	⋆	Correct vocabulary, a local calculation, or a one-rule translation.
Core	⋆⋆	A construction using several rules or a precise conceptual distinction.
Synthesis	⋆ ⋆ ⋆	Connection across operational, logical, implementation, or communication viewpoints.
Proof Workshop	⋆ ⋆ ⋆	A structured derivation or induction with the critical case made explicit.
Design Clinic	⋆ ⋆ ⋆⋆	A design plus a stated invariant, trust boundary, or failure mode.
Challenge	⋆⋆⋆⋆–⋆⋆⋆⋆⋆	Extension beyond the chapter, literature-facing reasoning, or a thesis/counterexample analysis.

Calibrated outliers

- Chapter 2, Proof Workshop 1 is closer to four-star for readers new to induction on derivations; accept a carefully annotated derivation-height proof.
- Chapter 4, Challenge 1 is closer to three-star once the de Bruijn convention is supplied; the research component, not the calculation, carries the additional difficulty.
- Chapter 9, Core 1 (predecessor) is a three-star construction in the particular nondependent recursor because the student must invent a product-valued state.
- Chapter 10, Challenge 1 (normalization-by-evaluation) and Chapter 7, Challenge 1 (classical control/Peirce) are genuine five-star reading problems unless the relevant machinery has been taught elsewhere.

- Chapter 13, Synthesis 2 becomes four-star if students are required to distinguish normalization of terms from termination of conversion/checking in a dependent language.
- Chapter 14, Challenge 1 is deliberately five-star: it is the volume's thesis-defense exercise and should be graded for explicit boundaries and counterexamples, not rhetorical confidence.

Chapter B

Instructor Rubric

For derivation problems, award credit separately for context formation, rule choice, premise derivations, and binding/freshness hygiene. For metatheory, require the induction measure and the critical constructor/eliminator cases; a slogan such as "by induction" is not full credit. For design exercises, require one explicit invariant and one failure mode. For synthesis/challenge essays, reward a claim whose scope is stated and stress-tested; penalize category errors such as equating type safety with termination or treating a context as literal time without an operational protocol semantics.

Companion Status

This Release Candidate 1 companion contains keyed responses for all 168 chapter exercises. Open-ended entries are reference solutions or rubrics; alternative correct constructions should be accepted when their typing, operational behavior, and stated assumptions are sound.